



Nombre y Apellido: [REDACTED]

Legajo: [REDACTED]

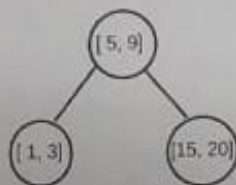
Examen Parcial 1

1. Encontrar una cota asintótica Θ para la siguiente recurrencia y verificar que la cota encontrada sea correcta.

$$\begin{aligned} T(1) &= a \\ T(n) &= 2 * T(n-1) + b \quad \text{si } n > 1 \end{aligned}$$

donde las constantes a y b son positivas.

2. Los árboles de intervalos fueron definidos para almacenar de manera eficiente conjuntos de enteros. Su definición se basa en la observación de que el conjunto $\{i \mid a \leq i \leq b\}$ puede representarse con el intervalo $[a, b]$. El siguiente es un ejemplo de un árbol de intervalo definido para el conjunto $\{1, 2, 3, 5, 6, 7, 8, 9, 15, 16, 17, 18, 19, 20\}$.



Un árbol de intervalo es un árbol binario t , tal que t es una hoja o t es un *Nodo* $l(a, b) r$ que cumple que

- a) $a \leq b$
- b) Si (x, y) es un valor de algún nodo de l entonces $y < a - 1$.
- c) Si (x, y) es un valor de algún nodo de r entonces $b + 1 < x$.
- d) l y r son árboles de intervalos.

Estos invariantes aseguran que los intervalos no se solapan ni se tocan y están ordenados de manera creciente en el árbol.

Se utilizará el siguiente tipo de datos para representar éstos árboles en Haskell:

```
type Interval = (Int, Int)
data ITree = E | N ITree Interval ITree
```

Definir las siguientes funciones en Haskell de manera eficiente:

- a) `right :: ITree -> Int`, que dado un árbol no vacío devuelva el extremo derecho del intervalo de más a la derecha del árbol.
Por ejemplo, `right (N (N E (1, 3) E) (5, 7) (N E (10, 12) E)) = 12`
- b) `checkIT :: ITree -> Bool` que dado un valor de tipo `ITree` chequee que éste sea un árbol de intervalo. Esta función puede definirse con trabajo en $O(2^h)$, donde h es la altura del árbol.
Dar la recurrencia correspondiente al trabajo de la función. (No resolverla)
- c) `splitMax :: ITree -> (Interval, ITree)`, que dado un árbol de intervalo t devuelva el intervalo que está más a la derecha en t (es decir, el que tiene su primer componente mayor a la del resto), y el árbol t sin éste elemento.
Por ejemplo, `splitMax (N (N E (1, 3) E) (5, 7) (N E (10, 12) E)) = ((10, 12), N (N E (1, 3) E) (5, 7) E)`
Se puede definir ésta función con trabajo en $O(h)$, siendo h la altura del árbol.
- d) Para eliminar un elemento del árbol, es necesario unir los árboles izquierdo y derecho de un árbol de intervalo. Para ello definir una función `merge :: ITree -> ITree -> ITree`, tal que si $N l r$ es un árbol de intervalo, `merge l r` devuelve un árbol de intervalo con los elementos de l y r . **Ayuda:** Usar la función del inciso anterior.
- e) `delElem :: ITree -> Int -> ITree`, que dado un árbol de intervalo y un entero elimine a éste del árbol. Para definir ésta función, se debe buscar primero el intervalo que contiene al elemento a eliminar y si éste existe considerar los siguientes casos: el intervalo es unitario, el elemento es uno de los extremos del intervalo o no.